

Trabajo Práctico Cuatrimestral

Intérprete con ANTLR

Objetivo

Diseñar e implementar un intérprete para un lenguaje de programación imperativo simple utilizando ANTLR como generador de lexer y parser. El trabajo debe incluir análisis léxico, análisis sintáctico, análisis semántico y ejecución de programas mediante un intérprete.

El trabajo deberá realizarse en grupos de 2 integrantes.

Todos los grupos compartirán un conjunto de características mínimas comunes. Además, cada grupo implementará una construcción diferencial asignada por el docente.

Lenguaje base

Cada grupo deberá diseñar un pequeño lenguaje de programación que incluya las capacidades descritas a continuación. La sintaxis concreta del lenguaje queda a criterio de cada grupo.

Tipos de datos

El lenguaje debe soportar al menos enteros, números reales, cadenas de texto y booleanos.

Comentarios

El lenguaje debe soportar comentarios de línea. Opcionalmente pueden agregarse comentarios multilínea. Los comentarios deben ser ignorados completamente por el intérprete.

Variables

El lenguaje debe permitir declarar variables, asignar valores y utilizar variables dentro de expresiones. Las variables no pueden utilizarse antes de ser declaradas y deben respetar las reglas de tipos definidas. La inicialización puede ser obligatoria u opcional.

Expresiones

El lenguaje debe soportar expresiones aritméticas, relacionales y lógicas. Como mínimo deben existir suma, resta, multiplicación, comparación y algún operador lógico.

Instrucción de salida

El lenguaje debe incluir alguna instrucción para imprimir resultados por consola.

Condicional if-else

El lenguaje debe incluir una estructura condicional con rama verdadera y falsa.

Ejemplo:

```
if (nota >= 6) {  
    print("Aprobado");  
} else {  
    print("Desaprobado");  
}
```

Construcciones diferenciales por grupo

Cada grupo implementará **sólo una** de las siguientes variantes, integrándola a su lenguaje. Las variantes deberán incluir validaciones semánticas y ejecución correcta dentro del intérprete. La variante a implementar será designada por sorteo.

Variante 1: while

Iteración con condición al inicio.

Ejemplo:

```
while (i < 5) {  
    print(i);  
    i = i + 1;  
}
```

Variante 2: for

Iteración con variable de control.

Ejemplo:

```
for (var i : int = 0; i < 5; i = i + 1) {  
    print(i);  
}
```

Variante 3: do-while

Iteración con condición al final.

Ejemplo:

```
do {  
    print(n);  
    n = n + 1;  
} while (n < 5);
```

Variante 4: repeat-until

Iteración con condición de corte invertida.

Ejemplo:

```
repeat {
```

```
    print(n);  
    n = n + 1;  
} until (n == 5);
```

Variante 5: switch

Selección múltiple por valor.

Ejemplo:

```
switch (dia) {  
    case 1:  
        print("Lunes");  
        break;  
    default:  
        print("Otro día");  
}
```

Análisis semántico

El proyecto debe incluir validaciones semánticas básicas. Se deben detectar uso de variables no declaradas, redeclaración de variables, incompatibilidades de tipos, operaciones inválidas y división por cero.

Resultado esperado

El programa desarrollado en Java debe ser capaz de leer un archivo fuente escrito en el lenguaje diseñado, analizarlo utilizando ANTLR4, detectar errores léxicos, sintácticos y semánticos, ejecutar correctamente programas válidos e imprimir resultados por consola.

El proyecto debe recorrer el árbol sintáctico generado por ANTLR utilizando Visitor.

Configuración del Proyecto

Estructura de carpetas sugerida

```
src/  
├── main/  
│   ├── antlr4/  
│   │   └── MiniLang.g4  
│   └── java/  
│       ├── Main.java  
│       ├── SymbolTable.java  
│       ├── SemanticAnalyzer.java  
│       └── Interpreter.java
```

Archivo pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>tp</groupId>
  <artifactId>tp-interpret</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <antlr.version>4.13.1</antlr.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.antlr</groupId>
      <artifactId>antlr4-runtime</artifactId>
      <version>${antlr.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>

      <plugin>
        <groupId>org.antlr</groupId>
        <artifactId>antlr4-maven-plugin</artifactId>
        <version>${antlr.version}</version>

        <configuration>
          <visitor>true</visitor>
          <listener>false</listener>
        </configuration>

        <executions>
          <execution>
            <goals>
              <goal>antlr4</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

```
        </execution>
    </executions>
</plugin>

<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.1.0</version>

    <configuration>
        <mainClass>Main</mainClass>
    </configuration>
</plugin>

</plugins>
</build>

</project>
```

Requisitos de entrega

La entrega deberá realizarse mediante un repositorio Git público o compartido con el docente. Alternativamente, podrá entregarse un archivo comprimido siempre que incluya la estructura completa del proyecto.

La entrega debe incluir el código fuente completo, la gramática .g4, el pom.xml, instrucciones de compilación y ejecución, y ejemplos de programas. No deben incluirse archivos generados automáticamente por ANTLR.

El README debe incluir los integrantes del grupo, la variante asignada, una descripción del lenguaje, las decisiones de diseño tomadas, instrucciones de compilación y ejecución y ejemplos de uso.

Criterio de evaluación

Se evaluará el correcto funcionamiento del análisis léxico y sintáctico, la calidad y claridad de la gramática definida, la implementación del análisis semántico y la detección adecuada de errores, el funcionamiento del intérprete y la correcta ejecución de programas válidos, así como la organización general del proyecto y la documentación entregada.

También se tendrá en cuenta la consistencia del diseño del lenguaje, la claridad de los mensajes de error y el uso adecuado de las herramientas provistas por ANTLR.